

ECPS204 Project2-Multi-threading canny edge

CHENG-CHIH, LEE, 29329351

JONATHAN KIM, 42411903

Project Description (2 pts)

- The purpose of this project is to implement multithreading on the canny application. OpenMP and pthread are explored and implemented.
- The improvement of performance is expected to be measured and discussed.

1. Experimental Setup (4 pts)

- For this project, we set up a Git repository to exchange files between the RPi and laptop. Following the instructions, we get each step done and recorded.
- The local canny file is used to study multithreading method. And the developed canny.cpp in proj1 is also modified to implement multithreading.

2. Results (6 pts)

Step 1 – Learning the basics of multi-thread programming with OpenMP

OpenMP is a simple way to use multiple CPU cores by adding instructions to existing code instead of rewriting it. OpenMP follows a fork-join model, where the program starts with one thread, creates multiple threads to execute parallel regions, and then joins back into a single thread. Parallel behavior is controlled mainly through `#pragma` instructions like `#pragma omp parallel`, `#pragma omp for`, and `#pragma omp critical`, which tell the compiler how work should be split and synchronized. OpenMP also provides runtime APIs, such as `omp_get_num_threads()` and `omp_get_thread_num()`, allowing basic control and tuning of thread execution at runtime while all threads share the same memory space.

Step 2 – Understanding the modifications on canny_local.c with OpenMP

canny_local_omp.c	canny_local_.c
<pre> 466 //***** 467 * Blur in the x - direction. 468 ****** 469 int n_thread = 4; //Edited by Jiang Wan for multi-threa 470 omp_set_dynamic(0); //Edited by Jiang Wan for multi-threa 471 omp_set_num_threads(n_thread); //Edited by Jiang Wan for multi-t </pre> In Blur_x, omp is implemented to create 4 threads. omp_set_dynamic(0) configured omp to not changing thread numbers dynamically unless limited by hardware resource.	<pre> 466 //***** 467 * Blur in the x - direction. 468 ****** 469 470 471 472 </pre> None
<pre> 472 #pragma omp parallel private(c, cc, dot, sum) //Edited by Jiang Wan for 473 { 474 if(VERBOSE) printf(" Blurring the image in the X-direction.\n"); </pre> Parallel variable copies are created for each thread. Otherwise, they will be shared.	None
<pre> 475 #pragma omp for 476 for(r=0;r<rows;r++){ 477 for(c=0;c<cols;c++){ 478 dot = 0; 479 sum = 0.0; 480 for(cc=(c-center);cc<=center;cc++){ 481 if(((c+cc) >= 0) && ((c+cc) < cols)){ 482 dot += (float)image[r*cols+(c+cc)] * kernel[center+cc]; 483 sum += kernel[center+cc]; 484 } 485 } 486 tempin[r*cols+c] = dot/sum; 487 } 488 } 489 490 </pre> #pragma omp for... dertermins the for loop to be run separately in each thread. “r” is copied here,	None

Step 3 – Compiling and running the modified code

```
cclee0@raspberrypi:~/Desktop/ECPS204/Embedded_SW_Course/WEEK2/src $ time ./canny_omp test.pgm 1.0 0.2 0.6
Reading the image test.pgm.
Starting Canny edge detection.
Smoothing the image using a gaussian kernel.
    Computing the gaussian smoothing kernel.
        The kernel has 7 elements.
The filter coefficients are:
kernel[0] = 0.004433
kernel[1] = 0.054006
kernel[2] = 0.242036
kernel[3] = 0.399050
kernel[4] = 0.242036
kernel[5] = 0.054006
kernel[6] = 0.004433
    Blurring the image in the X-direction.
    Blurring the image in the X-direction.
    Blurring the image in the X-direction.
    Blurring the image in the X-direction.
    Blurring the image in the Y-direction.
Computing the X and Y first derivatives.
    Computing the X-direction derivative.
    Computing the Y-direction derivative.
Computing the magnitude of the gradient.
Doing the non-maximal suppression.
Doing hysteresis thresholding.
The input low and high fractions of 0.200000 and 0.600000 computed to
magnitude of the gradient threshold values of: 155 774
Writing the edge image in the file test.pgm_s_1.00_l_0.20_h_0.60.pgm.

real    0m0.782s
user    0m1.006s
sys     0m0.032s
cclee0@raspberrypi:~/Desktop/ECPS204/Embedded_SW_Course/WEEK2/src $ time ./canny test.pgm 1.0 0.2 0.6

real    0m0.992s
user    0m0.902s
sys     0m0.020s
```

#canny_local_omp

```
real    0m0.782s
user    0m1.006s
sys     0m0.032s
```

#canny_local

```
real    0m0.992s
user    0m0.902s
sys     0m0.020s
```

From the results, we can see with multithreading implementation, the real run time is shortened. However, the user and system time increased. The real time is the wall time; the user time is the total cpu time and system time is the kernel time. With multithreading, the wall time is improved and cpu/system time increased due to multithread/core execution.

Step 4 – Use OpenMP to parallelize Blur in the y direction

```
492  .../*****
493  ... * Blur in the y - direction.
494  ... *****/
495  ...omp_set_dynamic(0); //Edited by CHENG-CHIH LEE for multi-thread
496  ...omp_set_num_threads(n_thread); //Edited by CHENG-CHIH LEE for multi-thread
497  ...#pragma omp parallel private(r, rr, dot, sum) //Edited by CHENG-CHIH LEE for multi-thread
498  ...{
499  ...    if(VERBOSE) printf("    Blurring the image in the Y-direction.\n");
500  ...    #pragma omp for
501  ...    for(c=0;c<cols;c++){
502  ...        for(r=0;r<rows;r++){
503  ...            sum = 0.0;
504  ...            dot = 0.0;
505  ...            for(rr=(-center);rr<=center;rr++){
506  ...                if(((r+rr) >= 0) && ((r+rr) < rows)){
507  ...                    dot += tempim[(r+rr)*cols+c] * kernel[center+rr];
508  ...                    sum += kernel[center+rr];
509  ...                }
510  ...            }
511  ...            (*smoothedim)[r*cols+c] = (short int)(dot*BOOSTBLURFACTOR/sum + 0.5);
512  ...        }
513  ...    }
514  ...}
515  ...free(tempim);
516  ...free(kernel);
517  ...}
```

OpenMP is implemented for Blur Y shown as above. In BlurY multithreading implementation, n_thread isn't declared again since it was done before BlurX. The variables to be copied are (r, rr, dot, sum).

tempim and kernel are freed after all the threads are done execution.

```
cclee@raspberrypi:~/Desktop/ECP5204/Embedded_SW_Course/WEEK2/src $ time ./canny_omp test.pgm 1.0 0.2 0.6
Reading the image test.pgm.
Starting Canny edge detection.
Smoothing the image using a gaussian kernel.
    Computing the gaussian smoothing kernel.
        The kernel has 7 elements.
The filter coefficients are:
kernel[0] = 0.004433
kernel[1] = 0.054006
kernel[2] = 0.242036
kernel[3] = 0.399050
kernel[4] = 0.242036
kernel[5] = 0.054006
kernel[6] = 0.004433
    Blurring the image in the X-direction.
    Blurring the image in the X-direction.
    Blurring the image in the X-direction.
    Blurring the image in the X-direction.
    Blurring the image in the Y-direction.
    Blurring the image in the Y-direction.
    Blurring the image in the Y-direction.
    Blurring the image in the Y-direction.
Computing the X and Y first derivatives.
    Computing the X-direction derivative.
    Computing the Y-direction derivative.
Computing the magnitude of the gradient.
Doing the non-maximal suppression.
Doing hysteresis thresholding.
The input low and high fractions of 0.200000 and 0.600000 computed to
magnitude of the gradient threshold values of: 155 774
Writing the edge image in the file test.pgm_s_1.00_l_0.20_h_0.60.pgm.

real    0m0.662s
user    0m0.913s
sys     0m0.044s
```

From this snapshot, we can see there are also 4 threads created for Blur Y.

And from the time result below, we can tell the real time is improved by approximately **33%** compared to the original application without multithreading and **15 %** compared to the application which only implemented multithreading in BlurX.

```
real    0m0.662s
user    0m0.913s
sys     0m0.044s
```

Step 5 – Learning the basics of multi-thread programming with pthread

POSIX threads provide a low-level way to implement parallelism by manually creating and managing threads within a single process. Like OpenMP, pthreads use a shared-memory model, meaning all threads run in the same address space and can access the same global and heap memory.

Each thread starts execution in a user-defined function and receives its own argument structure, allowing the programmer to control whether a data is shared or private. Unlike OpenMP's pragma-based approach, pthreads require the programmer to handle thread creation, workload partitioning, and synchronization directly using APIs such as `pthread_create()`, `pthread_join()`, and mutexes, offering more control but also more implementation complexity.

Step 6 – Understanding the modifications on canny_local.c with pthread

canny_local_pthread.c	canny_local.c
<pre> 429 //Added by Jiang Wan for multi-thread 430 struct thread_args_x 431 { 432 unsigned char *image; 433 int rows; 434 int cols; 435 int col_s; 436 int col_e; 437 int center; 438 float *kernel; 439 float *tempim; 440 }; </pre> Structure is created for passing arguments for multithreading	None
<pre> 441 //Added by Jiang Wan for multi-thread 442 void *blur_x(void *arguments) 443 { 444 //***** 445 // Blur in the x - direction. 446 //***** 447 struct thread_args_x *args = arguments; 448 unsigned char *image = args->image; 449 int rows = args->rows; 450 int cols = args->cols; 451 int col_s = args->col_s; 452 int col_e = args->col_e; 453 int center = args->center; 454 float *kernel = args->kernel; 455 float *tempim = args->tempim; 456 int r, c, cc; 457 float dot, sum; 458 if(DEBUG) printf(" Blurring the image in the X-direction.\n"); 459 for(c=col_s; c<col_e; c++){ 460 dot = 0.0; 461 sum = 0.0; 462 for(cc=center; cc<center+cols; cc++){ 463 if((c+cc) >= 0) && (c+cc) < cols){ 464 dot += ((float)image[rows*(c+cc)] * kernel[center+cc]); 465 sum += kernel[center+cc]; 466 } 467 } 468 tempim[rows*c] = dot/sum; 469 } 470 } </pre> blur_x is modified to take argument and have local variables declared with init value from the argument,	None
<pre> 478 //Edited by Jiang Wan for multi-thread 479 #define N_T 4 </pre> Number of thread is defined as 4	None
<pre> 517 pthread_t thread[N_T]; 518 int iret[N_T]; 519 struct thread_args_x args[N_T]; 520 521 int col_per_t = cols/N_T; 522 int i; 523 for(i=0; i<N_T; i++){ 524 { 525 args[i].image = image; 526 args[i].rows = rows; 527 args[i].cols = cols; 528 args[i].col_s = col_per_t*i; 529 args[i].col_e = col_per_t*(i+1); 530 args[i].center = center; 531 args[i].kernel = kernel; 532 args[i].tempim = tempim; 533 iret[i] = pthread_create(&thread[i], NULL, &blur_x, &args[i]); 534 } 535 } </pre> In gaussian smooth, threads are created for BlurX and executed	None
<pre> 537 for(i=0; i<N_T; i++) 538 { 539 pthread_join(thread[i], NULL); 540 } 541 </pre> Each thread is waited to join before process moves on.	None

Step 7 – Compiling and running the modified code with pthreads

```
cc@raspberrypi:~/Desktop/ECPS204/Embedded_SW_Course/WEEK2/src $ time ./canny_thread test.pgm 1.0 0
Reading the image test.pgm.
Starting Canny edge detection.
Smoothing the image using a gaussian kernel.
    Computing the gaussian smoothing kernel.
        The kernel has 7 elements.
The filter coefficients are:
kernel[0] = 0.004433
kernel[1] = 0.054006
kernel[2] = 0.242036
kernel[3] = 0.399050
kernel[4] = 0.242036
kernel[5] = 0.054006
kernel[6] = 0.004433
    Blurring the image in the X-direction.
    Blurring the image in the X-direction.
    Blurring the image in the X-direction.
    Blurring the image in the X-direction.
    Blurring the image in the Y-direction.
Computing the X and Y first derivatives.
    Computing the X-direction derivative.
    Computing the Y-direction derivative.
Computing the magnitude of the gradient.
Doing the non-maximal suppression.
Doing hysteresis thresholding.
The input low and high fractions of 0.200000 and 0.600000 computed to
magnitude of the gradient threshold values of: 155 774
Writing the edge image in the file test.pgm_s_1.00_l_0.20_h_0.60.pgm.

real    0m0.852s
user    0m0.954s
sys     0m0.032s
```

#canny_pthread

```
real    0m0.852s
user    0m0.954s
sys     0m0.032s
```

#canny_local_omp

```
real    0m0.782s
user    0m1.006s
sys     0m0.032s
```

#canny_local

```
real    0m0.992s
user    0m0.902s
sys     0m0.020s
```

From the results, we can tell the implementation of pthread did improve the wall time, but not as much as OpenMP.

Step 8 – Modifying the Blur in the y - direction with pthreads

```
430 struct thread_args_x
431 {
432     unsigned char *image;
433     int rows;
434     int cols;
435     int col_s;
436     int col_e;
437     int row_s;
438     int row_e;
439     int center;
440     float *kernel;
441     float *tempim;
442     short int *smoothedim;
443 };
```

Arguments added for Blur Y use.

```
481 //Added by CHENH-CHIH LEE for multi-thread
482 void *blur_y(void *arguments)
483 {
484     /*
485     * Blur in the y - direction.
486     */
487     struct thread_args_x *args = arguments;
488     unsigned char *image = args->image;
489     int rows = args->rows;
490     int cols = args->cols;
491     int row_s = args->row_s;
492     int row_e = args->row_e;
493     int center = args->center;
494     float *kernel = args->kernel;
495     float *tempim = args->tempim;
496     short int *smoothedim = args->smoothedim;
497     int r, c, rr;
498     float dot, sum;
499     if(VERBOSE) printf("...Blurring the image in the Y-direction.\n");
500     for(r=row_s; r<row_e; r++){
501         for(c=0; c<cols; c++){
502             sum = 0.0;
503             dot = 0.0;
504             for(rr=(-center); rr<=center; rr++){
505                 if(((r+rr) >= 0) && ((r+rr) < rows)){
506                     dot += tempim[(r+rr)*cols+c] * kernel[center+rr];
507                     sum += kernel[center+rr];
508                 }
509             }
510             (smoothedim)[r*cols+c] = (short int) (dot*BOOSTBLURFACTOR/sum + 0.5);
511         }
512     }
513     return NULL;
514 }
```

Blur Y seperated for multithreading. Iteration on rows, so we modified the for loops as cols in the inner loop.

```
578 for(i=0; i<N_T; i++)
579 {
580     pthread_join(thread[i], NULL);
581 }
582 //Thread creation for Blur Y
583 int row_per_t = rows/N_T;
584 for(i=0; i<N_T; i++)
585 {
586     args[i].image = image;
587     args[i].rows = rows;
588     args[i].cols = cols;
589     args[i].row_s = row_per_t*i;
590     args[i].row_e = row_per_t*(i+1);
591     args[i].center = center;
592     args[i].kernel = kernel;
593     args[i].tempim = tempim;
594     args[i].smoothedim = *smoothedim;
595     pthread_t[i] = pthread_create(&thread[i], NULL, &blur_y, &args[i]);
596 }
597 for(i=0; i<N_T; i++)
598 {
599     pthread_join(thread[i], NULL);
600 }
601 }
```

Threads created for Blur Y.


```

cc@raspberrypi:~/Desktop/ECPS204/Embedded_SW_course/WEEK2/src $ time ./canny_pthread test.pgm 1.0 0.2 0.6
Reading the image test.pgm.
Starting Canny edge detection.
Smoothing the image using a gaussian kernel.
  Computing the gaussian smoothing kernel.
    The kernel has 7 elements.
The filter coefficients are:
kernel[0] = 0.004433
kernel[1] = 0.054006
kernel[2] = 0.242036
kernel[3] = 0.399050
kernel[4] = 0.242036
kernel[5] = 0.054006
kernel[6] = 0.004433
  Blurring the image in the X-direction.
  Blurring the image in the X-direction.
  Blurring the image in the X-direction.
  Blurring the image in the X-direction.
  Blurring the image in the Y-direction.
  Blurring the image in the Y-direction.
  Blurring the image in the Y-direction.
  Blurring the image in the Y-direction.
Computing the X and Y first derivatives.
  Computing the X-direction derivative.
  Computing the Y-direction derivative.
Computing the magnitude of the gradient.
Doing the non-maximal suppression.
Doing hysteresis thresholding.
The input low and high fractions of 0.200000 and 0.600000 computed to
magnitude of the gradient threshold values of: 155 774
Writing the edge image in the file test.pgm_s_1.00_1_0.20_h_0.60.pgm.

real    0m0.530s
user    0m0.779s
sys     0m0.032s

```

Now from the snapshot, we can see there are 4 threads created for Blur Y.

Time results are below:

#canny_pthread X+Y

real 0m0.530s

user 0m0.779s

sys 0m0.032s

#canny_local_omp

real 0m0.662s

user 0m0.913s

sys 0m0.044s

#canny_local

real 0m0.992s

user 0m0.902s

sys 0m0.020s

We can see the wall time is improved and having better performance than omp.

Step 9 – Comparing OpenMP and pthreads

	Original	OMP	OMP to Original	pThread	pThread to Original
real	0m0.992s	0m0.662s	Improve 33.3%	0m0.530s	Improve 46.6%
user	0m0.902s	0m0.913s	Deteriorate 1.2%	0m0.779s	Improve 13.6%
sys	0m0.020s	0m0.044s	Deteriorate 120%	0m0.032s	Deteriorate 60%

From the comparison, we can see that OpenMP is bringing large overhead to the kernel loading. And pthread is also resulting in better WALL time improvement.

Step 10 - Applying multi-threading (in OpenMP and pthread individually) to the camera-capturing-and-edge-detection system (in Proj #1-b)

#150 frames were taken in both OMP and pthread implementations.

1. Implementing **OpenMP**

i. Snapshot

```
cclee@raspberrypi:~/Desktop/ECPS204/Embedded_SW_Course/WEEK2/src/camera_canny_omp $ ./camera_canny_omp 1.5 0.3 0.6 n 150 camera_canny_img
[5:32:07.716889132] [3520] INFO Camera camera_manager.cpp:330 libcamera v0.5.2+99-bfd68f78
[5:32:07.723878533] [3526] INFO RPI pisp.cpp:720 libpisp version v1.2.1 981977ff21f3 29-04-2025 (14:13:50)
[5:32:07.738067111] [3526] INFO IPAProxy ipa_proxy.cpp:180 Using tuning file /usr/share/libcamera/ipa/rpi/pisp/ov5647.json
[5:32:07.745366375] [3526] INFO Camera camera_manager.cpp:220 Adding camera '/base/axi/pcie@120000/rp1/i2c@88000/ov5647@36' for pipeline ha
[5:32:07.745396524] [3526] INFO RPI pisp.cpp:1179 Registered camera /base/axi/pcie@120000/rp1/i2c@88000/ov5647@36 to CFE device /dev/media0
t12.60
Saved camera_canny_img/frame147.pgm | CPU capture=0.001654 s, CPU process=0.081673 s
Saved camera_canny_img/frame148.pgm | CPU capture=0.001646 s, CPU process=0.083112 s
Saved camera_canny_img/frame149.pgm | CPU capture=0.007634 s, CPU process=0.076910 s
Saved camera_canny_img/frame150.pgm | CPU capture=0.002301 s, CPU process=0.083702 s

===== SUMMARY =====
Frames processed: 150
Total WALL time (steady_clock): 9.381065 s
Total CPU time (clock): 14.075344 s
Total CPU capture time: 0.372759 s
Total CPU process time: 12.346313 s
Avg FPS (WALL): 15.989655
Avg FPS (CPU): 10.656933
Output folder: camera_canny_img
```

ii. Performance

```
===== SUMMARY =====
Frames processed: 150
Total WALL time (steady_clock): 9.237100 s
Total CPU time (clock): 14.016798 s
Total CPU capture time: 0.315001 s
Total CPU process time: 12.469819 s
Avg FPS (WALL): 16.238863
Avg FPS (CPU): 10.701446
Output folder: camera_canny_img
=====
```

2. Implementing **pthread**

i. Snapshot

```
cclee@raspberrypi:~/Desktop/ECPS204/Embedded_SW_Course/WEEK2/src/camera_canny_pt $ ./camera_canny_pt 1.5 0.3 0.6 n 150 camera_canny_img
[6:16:23.216108560] [3788] INFO Camera camera_manager.cpp:330 libcamera v0.5.2+99-bfd68f78
[6:16:23.223825602] [3795] INFO RPI pisp.cpp:720 libpisp version v1.2.1 981977ff21f3 29-04-2025 (14:13:50)
[6:16:23.223825602] [3795] INFO IPAProxy ipa_proxy.cpp:180 Using tuning file /usr/share/libcamera/ipa/rpi/pisp/ov5647.json
Saved camera_canny_img/frame149.pgm | CPU capture=0.000762 s, CPU process=0.060352 s
Saved camera_canny_img/frame150.pgm | CPU capture=0.001584 s, CPU process=0.062941 s

===== SUMMARY =====
Frames processed: 150
Total WALL time (steady_clock): 8.543042 s
Total CPU time (clock): 10.724717 s
Total CPU capture time: 0.324200 s
Total CPU process time: 9.010140 s
Avg FPS (WALL): 17.558149
Avg FPS (CPU): 13.986383
Output folder: camera_canny_img
```

ii. Performance

```
===== SUMMARY =====
Frames processed: 150
Total WALL time (steady_clock): 8.543042 s
Total CPU time (clock): 10.724717 s
Total CPU capture time: 0.324200 s
Total CPU process time: 9.010140 s
Avg FPS (WALL): 17.558149
Avg FPS (CPU): 13.986383
Output folder: camera_canny_img
=====
```

3. Implementation from Proj1

i. Snapshot

```
cclee0@raspberrypi:~/Desktop/ECPS204/Embedded_SW_Course/WEEK1/src $ ./camera_canny 1.5 0.3 0.6 n 150 camera_canny_img
[6:28:36.795717693] [5169] INFO camera camera_manager.cpp:330 libcamera v0.5.2+09-bfd68f78
[6:28:36.803383383] [5175] INFO RPI pisp.cpp:720 libpisp version v1.2.1 981977ff21f3 29-04-2025 (14:13:50)
Saved camera_canny_img/frame149.pgm | CPU capture=0.002317 s, CPU process=0.070970 s
Saved camera_canny_img/frame150.pgm | CPU capture=0.002331 s, CPU process=0.070700 s

===== SUMMARY =====
Frames processed: 150
Total WALL time (steady_clock): 12.236731 s
Total CPU time (clock): 12.270120 s
Total CPU capture time: 0.346681 s
Total CPU process time: 10.730583 s
Avg FPS (WALL): 12.258176
Avg FPS (CPU): 12.224819
Output folder: camera_canny_img
=====
```

ii. Performance

```
===== SUMMARY =====
Frames processed: 150
Total WALL time (steady_clock): 12.236731 s
Total CPU time (clock): 12.270120 s
Total CPU capture time: 0.346681 s
Total CPU process time: 10.730583 s
Avg FPS (WALL): 12.258176
Avg FPS (CPU): 12.224819
Output folder: camera_canny_img
=====
```

4. Performance Comparison

	Proj1	OMP	Impv. OMP to Proj1	pThread	Impv. pThread to Proj1
Total WALL time	12.236731 s	9.237100 s	Impv. 24.5%	8.543042 s	Impv. 30.2%
Total CPU time (clock)	12.270120 s	14.016798 s	Detero. 14.2%	10.724717 s	Impv. 12.6%
Total CPU capture time	0.346681 s	0.315001 s	Impv. 9.1%	0.324200 s	Impv. 6.5%
Total CPU process time	10.730583 s	12.469819 s	Detero. 16.2%	9.010140 s	Impv. 16.1%
Avg FPS (WALL)	12.258176	16.238863	Impv. 32.5%	17.558149	Impv. 43.3%
Avg FPS (CPU)	12.224819	10.701446	Detero. 12.5%	13.986383	Impv. 14.4%

From our implementation and testing, we noticed that OMP performs worse than pthread, which is quite identical in the previous “local” single image application. However, we get WALL time improvement in both implementations.

3. Problems and Discussion (6 pts)

- OpenMP is rather easy to implement, thus bringing no issue.
- POSIX requires us to modify the source code heavily. We faced issue in adding argument “smoothedim” to the structure regarding type casting. However, it was just a pointer casting issue, so we fixed it quickly.
- After modifying the pthread implementation code, the first-round test failed unexpectedly. The root cause is not clear now since we couldn’t reproduce it. Our guess is that a segmentation fault happened.

4. Conclusion (2 pts)

- In this project, two multithreading methods are explored with both implementations done.
- Performance comparison is done through the time measurement approach from project 1.
- OpenMP is easier to implement, whereas the performance improvement is not as good as POSIX. Developers should take the project development status and project requirement into consideration upon choosing the approach.